

CSAI 498 / CSAI 499 – Graduation Project Proposal

Cover Page

- Project Title: **Adaptive Wardens**

	Student Name	ID	Program
	Abdulkhaliq Sarwat	202202084	IT
	Ali Nazeer	202100732	IT
	Ahmed Yasser	202201883	IT
	Ali Ahmed reda	202201006	IT

- Supervisor: Dr. Ashraf Hafez Badawi
- Semester / Year: Fall / 2025
- Date of Submission: 25/10/2025

Abstract

Traditional honeypots are easy to fingerprint because they reply with static response (script) , **repetitive**, **scripted** responses and do not maintain believable system state. We propose an Adaptive AI-Driven Honeypot that uses a small transformer model and a sandboxed state store to generate context-aware, stateful, and variable responses for SSH and HTTP interactions. The honeypot will simulate **realistic** filesystems, processes, timing, and occasional errors while extracting Indicators of Compromise (IOCs) and **mapping** attacker behavior to MITRE ATT&CK. Expected outcomes: **longer** attacker engagement , **lower detection** probability, and **higher-quality threat intelligence** suitable for SOC ingestion and research.

Problem Statement & Motivation → (SO 1)

- **Problem.** Current **low-cost** honeypots are largely **scripted** and **stateless**. Attackers quickly identify them and leave, producing low-value telemetry and limited insight into attacker **tactics**, **techniques**, and **procedures** (TTPs).

Why does it matter?

- Organizations lose the chance to observe attacker interaction and behaviours in realistic interactive sessions.

- SOCs and research teams receive noisy, low-context alerts (IP lists) instead of actionable TTP mappings.
- Attackers can quickly know that they are in a honeypot due to the known behavior of the static scripts

Who is affected?

- Security operations centers (SOCs), CERTs, academic researchers, and small/medium enterprises that depend on **deception** tech for early detection and intel.

Motivation

Build an inexpensive, safe, privacy-respecting honeypot that feels convincingly real to an adversary long enough to **collect high-value intelligence** and to support defensive research and rule engineering

Proposed Solution → (SO 1 & SO 2)

- A modular system consisting of:

1. AI Engine

- Accepts incoming attacker commands (SSH shell lines, HTTP requests), returns plausible OS-style output and action results (simulated file contents, process lists).
- Performs Named Entity Recognition (IPs, domains, hashes, tool names) and maps events to MITRE ATT&CK techniques.

2. Sandbox State Store

- JSON/SQLite representation of a **fake filesystem**, users, processes, bash history, logs, and service state.
- Mutations occur only in the sandbox; no real OS commands are executed. State persists across sessions (configurable TTL).

3. Service Emulators / Frontends

- **SSH frontend:** lightweight SSH server that **proxies** commands (no shell escape) to **AI Engine**. Accepts login attempts, simulates password files and system responses.
- **HTTP frontend:** web server that responds to **probing** and **payloads** with AI-generated web page contents and route behavior.

4. IOC Extraction & ATT&CK Mapping

- Real-time **extraction** of **IOCs** (IPs, file names, payloads, tool signatures).
- Heuristic + model **mapping** to ATT&CK techniques with confidence scores.

5. Dashboard

- Live sessions, session playback, IOCs summary, ATT&CK heatmap, comparison charts (**baseline** vs **AI**).
- Exportable reports and JSON artifacts for SOC ingestion.

Key differentiators vs traditional honeypots.

- **Persistent** fake state for session continuity.
- **Controlled** realism (timings, errors, banners) to reduce **fingerprintability**.
- **IOC extraction** and ATT&CK mapping for high-value intel.

Project Scope → (SO 2)

- In Scope

- Implement SSH and HTTP emulators that forward attacker **inputs to the AI Engine**.
- Build the AI Engine
- Implement the Sandbox State Store
- Real-time IOC extraction and ATT&CK mapping component.

- Dashboard.
- Automated evaluation against a baseline scripted honeypot

Out of Scope

- Executing any **real** commands on the host OS (no real shell execution).
- Production deployment in customer networks or internet-facing honeypots (**unless authorized and approved**).

High-Level Timeline → (SO 2)

Phase	Description	Duration (weeks)	Deliverables
Research & Requirement Analysis	Study related work, define objectives, gather requirements, and identify tools, datasets, and evaluation criteria.	4 weeks	Requirement document, project plan, literature summary
Design & Planning	Design the overall system architecture, outline main components and data flow, and prepare detailed implementation plan and timeline.	4 weeks	System design document, architecture diagrams, finalized implementation plan
Implementation Part 1	Develop the core system modules and integrate main functionalities according to the project design.	9 weeks	Working prototype with core features implemented

Implementation Part 2 / Integration	Complete system integration, add remaining components, and prepare the system for testing and evaluation.	4 weeks	Fully integrated prototype ready for testing
Testing & Evaluation	Test system performance and reliability, analyze results, and prepare the final report and presentation.	4 weeks	Evaluation report, final documentation, and presentation

Technology Stack & Theoretical Basis → (SO 6 – Program Specific)

- **Languages:** Python TypeScript (React dashboard)
Justification: Python will be used in **ML** (Python) and for **FastAPI**; TypeScript + React for a robust, maintainable frontend.
- **AI / ML:** Python Libraries
Justification: GPU support
- **Web/API:** FastAPI (AI Engine + Dashboard backend)
Justification: FastAPI provides easy integration with Python ML code.
- **Frontend:** React + TypeScript (dashboard)
Justification: **Modern UI**
- **State Store / DB:** SQLite (sandbox state)
Justification: SQLite simple to ship and checkpoint
- **Dev Tools:** GitHub

Success Metrics & Evaluation Plan → (SO 2 & SO 6)

Quantitative metrics (will be collected during controlled tests):

1. **Average session length** — compare AI honeypot vs baseline scripted honeypot.
2. **Commands per session** — higher = deeper engagement.
3. **Time until detection / abandonment** — longer time indicates better believability.
4. **Unique IOCs extracted per session** — richness of telemetry.
5. **Coverage of ATT&CK techniques detected** — breadth of captured TTPs.
6. **False positive rate of ATT&CK mapping** — measure mapping precision.
7. **System performance**: average response **latency** per request (should be realistic, not too fast).

Team Roles & Responsibilities → (SO 5 & SO 3)

Team Member	Program	Primary Role	Technical Contribution (SO 6 Focus)
Ali Ahmed Reda	IT	AI & IOC Engine	● Implement transformer model for context-aware responses ● Parse incoming commands, generate plausible outputs ● Extract IOCs and map to ATT&CK ● Integrate AI with sandbox
Abdulkhaliq Sarwat	IT	SSH/HTTP Frontend & Emulators	● Implement SSH server and HTTP server frontends ● Handle incoming connections and forward to AI engine ● Simulate system responses without executing real commands
Ahmed Yasser	IT	Sandbox & State Store	● Design JSON/SQLite sandbox ● Maintain fake filesystem, processes, logs ● Ensure state persistence across sessions

			• Provide API for AI engine and frontends to read/write state
Ali Nazeer	IT	Dashboard & Evaluation	• Build React + TypeScript dashboard • Display live sessions, session playback, IOC summary, ATT&CK mapping • Collect quantitative metrics (session length, commands/session, detection time) • Generate reports & JSON exports

Communication / Coordination

- **Meetings:** 2–3x per week, 30–45 mins, short progress updates.
- **Tools:** Discord / Teams for daily chat, GitHub for code & issues, Google Docs or Overleaf for docs.
- **Integration:** Frontend and backend link weekly to make sure AI engine, sandbox, and dashboards work together.

References

[1] N. Gaddam, “AI-Enhanced Honeypots for Advanced Cyber Deception Strategies,” *QITP-IJCSRD: International Journal of Cyber Security Research and Development*, vol. 5, no. 1, pp. 9–19, Jan.–Jun. 2025. [Online]. Available: https://gitpress.com/articles/QITP-IJCSRD/VOLUME_5_ISSUE_1/QITP-IJCSRD_05_01_002.pdf

[2] S. A. Kareem, R. C. Sachan, and R. K. Malviya, “AI-Driven Adaptive Honeypots for Dynamic Cyber Threats,” *SSRN Electronic Journal*, Sept. 17, 2024. [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4966935

Github Repository:

[Aligithyb/-AdaptiveWardens](#)